

LAB SHEET

06026212 DATA WAREHOUSING

สัปดาห์ที่ 5: การวิเคราะห์ ออกแบบ และสร้าง
Snowflake Schema ด้วย dbt

ระยะเวลา 2 ชั่วโมง | PostgreSQL + dbt + Metabase + Docker

วัตถุประสงค์

- 1. วิเคราะห์และออกแบบ Snowflake Schema จาก coffee_sales.csv ได้
- 2. ใช้ dbt seed โหลด coffee_sales.csv และ province_region_mapping.csv เข้าสู่ PostgreSQL ได้
- 3. สร้าง staging models, dimension tables, fact table และ reporting view ด้วย ref() ได้
- 4. ใช้ hierarchy_def เป็น metadata ควบคุมลำดับชั้น region → province ได้
- 5. ใช้ dbt test และ dbt docs ตรวจสอบคุณภาพและ lineage ของโมเดลได้
- 6. สร้าง Dashboard บน Metabase ที่เลือกแสดงรายได้ตามระดับ hierarchy ได้

เครื่องมือและชุดข้อมูล

เครื่องมือ	หน้าที่
PostgreSQL	ฐานข้อมูลปลายทางใน Docker
dbt-postgres	โหลด seed, แปลงข้อมูล, ทดสอบ และสร้าง documentation
Metabase	สร้างคำถามและ Dashboard จาก reporting view
Docker Compose	เปิดบริการ PostgreSQL, dbt และ Metabase
VS Code / Text editor	สร้างและแก้ไขไฟล์โครงการ dbt

- coffee_sales.csv — 3,000 รายการขาย ใช้ชุดเดียวกับ Lab 4
- province_region_mapping.csv — 3 จังหวัด พร้อม region สำหรับสร้างลำดับชั้นภูมิศาสตร์

ส่วนที่ 1 วิเคราะห์และออกแบบ Snowflake Schema

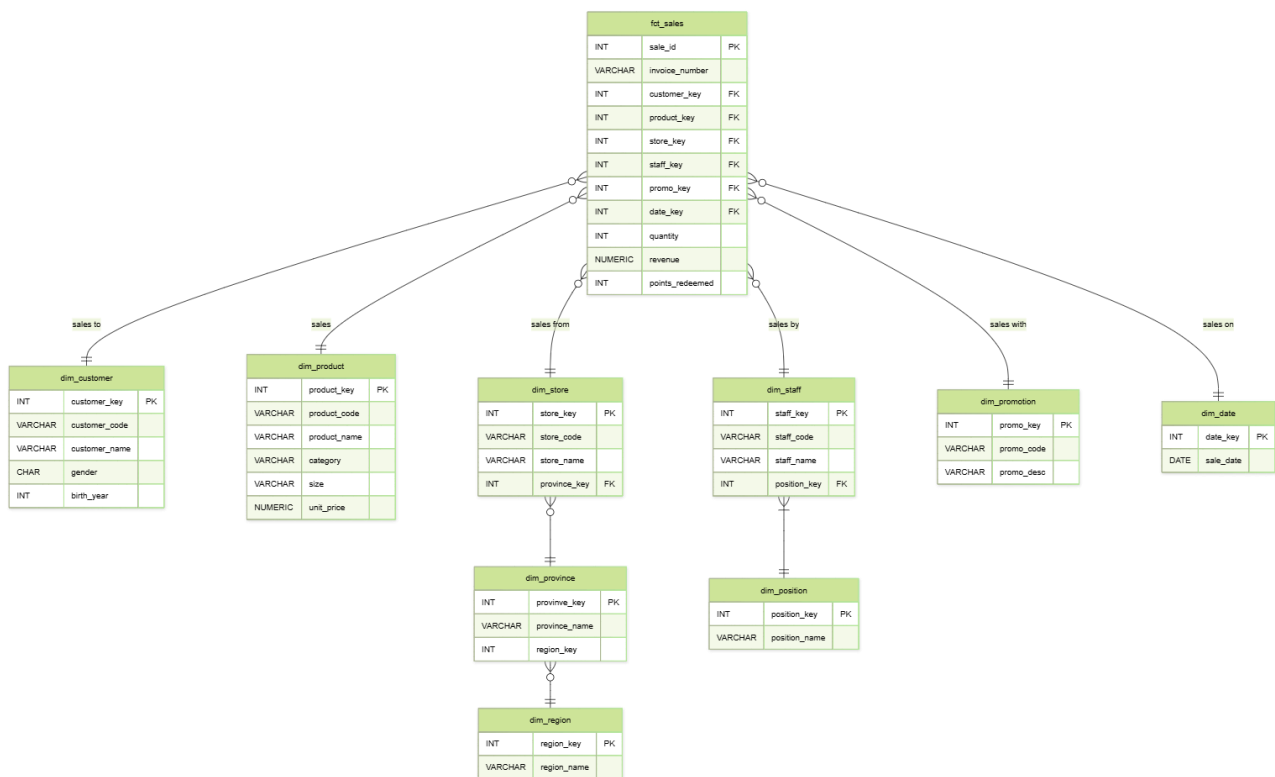
1.1 ระบุ grain และ hierarchy

Fact grain: หนึ่งแถวใน fact_sales แทนหนึ่งรายการขาย โดยใช้ sale_id เป็นตัวระบุรายการ

- Geography: dim_region → dim_province → dim_store
- Product: dim_category → dim_product
- Staff: dim_position → dim_staff
- Dimensions ที่ไม่แตกแขนง: dim_customer, dim_date และ dim_promotion

สังเกต: Snowflake Schema ทำให้ข้อมูลระดับบน เช่น region, category และ position ไม่ถูกเก็บซ้ำในมิติระดับล่าง แต่ query จะต้อง join หลายตารางขึ้น

1.2 ER-Diagram เบ้าหมาย



ภาพที่ 1 โครงสร้าง Snowflake Schema ที่ dbt จะสร้าง

dbt กับ Foreign Key: Lab นี้ใช้ relationship tests ตรวจสอบความสมบูรณ์ของคีย์อ้างอิง แผนการสร้าง physical FOREIGN KEY constraint โดยตรง จึงตรวจจับ orphan records ได้และยังคง transformation แบบ declarative

ส่วนที่ 2 เตรียม Lab Environment

2.1 สร้างฐานข้อมูลผ่าน pgAdmin SQL Editor

```
CREATE DATABASE coffee_dw_snowflake;
```

2.2 สร้างโครงสร้างโครงการ

```

dbt_root/
├── profiles.yml
└── dbt/
    ├── coffee_dw_snowflake
    │   ├── dbt_project.yml
    │   ├── seeds/
    │   │   ├── coffee_sales.csv
    │   │   └── province_region_mapping.csv
    │   ├── models/
    │   │   ├── coffee_dw/
    │   │   │   ├── staging/
    │   │   │   ├── marts/
    │   │   │   ├── metadata/
    │   │   │   ├── reporting/
    │   │   │   └── schema.yml
    │   │   └── tests/
    
```

คัดลอก coffee_sales.csv และ province_region_mapping.csv ไปไว้ในโฟลเดอร์
coffee_dw_snowflake/seeds/ โดยคงชื่อไฟล์เดิม

2.3 กำหนด profile และ project configuration

dbt_root/profiles.yml

```

coffee_dw_snowflake:
  target: dev
  outputs:
    dev:
      type: postgres
      host: postgres
      port: 5432
      user: dw_user
      password: dw_pass
      dbname: coffee_dw_snowflake
      schema: dbt
      threads: 4
  
```

ข้อ host: ภายใน Docker network ต้องใช้ชื่อ service คือ postgres ไม่ใช่ localhost ส่วน dw_postgres เป็น container_name ซึ่งใช้ได้ในบางกรณี

dbt/coffee_dw_snowflake/dbt_project.yml

```

name: coffee_dw_snowflake
version: '1.0.0'
config-version: 2

profile: coffee_dw_snowflake

model-paths: ["models"]
seed-paths: ["seeds"]
test-paths: ["tests"]

target-path: "target"
clean-targets:
  - "target"
  - "dbt_packages"

models:
  coffee_dw_snowflake: # project name
    staging:
      +materialized: view
      +schema: staging
    marts:
      +materialized: table
      +schema: marts
    metadata:
      +materialized: table
      +schema: metadata
    reporting:
      +materialized: view
      +schema: reporting

seeds:
  coffee_dw_snowflake:
    +schema: raw
  coffee_sales:
    +column_types:
  
```

```

sale_id: integer
sale_date: date
birth_year: integer
unit_price: numeric(10,2)
quantity: integer
revenue: numeric(12,2)
points_redeemed: integer
province_region_mapping:
+column_types:
  province_name: varchar(100)
  region_name: varchar(100)

```

2.4 ตรวจสอบการเชื่อมต่อ

dbt debug

ผลที่ได้: Connection test: OK และ All checks passed

ส่วนที่ 3 โหลดข้อมูลด้วย dbt seed

dbt seed อ่าน CSV ในโฟลเดอร์ seeds และสร้างตารางใน schema coffee_dw_raw ตามชนิดข้อมูลที่กำหนดใน dbt_project.yml

dbt seed

Seed	จำนวนแถวที่คาดหวัง	Schema
coffee_sales	3,000	coffee_dw_raw
province_region_mapping	3	coffee_dw_raw

รับข้อหลังแก้ CSV: ใช้ dbt seed --full-refresh เพื่อสร้าง seed table ใหม่จากไฟล์ล่าสุด

ส่วนที่ 4 สร้าง Staging Models

Staging layer ทำหน้าที่ cast ชนิดข้อมูล, trim ช่องว่าง, ปรับ category เป็นตัวพิมพ์เล็ก และเปลี่ยนค่าว่างของ promotion ให้เป็น NULL

models/staging/stg_coffee_sales.sql

```

select
  cast(sale_id as integer) as sale_id,
  trim(invoice_number) as invoice_number,
  cast(sale_date as date) as sale_date,
  trim(customer_code) as customer_code,
  trim(customer_name) as customer_name,
  trim(gender) as gender,
  cast(birth_year as integer) as birth_year,
  trim(product_code) as product_code,
  trim(product_name) as product_name,
  lower(trim(category)) as category,
  trim(size) as size,
  cast(unit_price as numeric(10,2)) as unit_price,
  cast(quantity as integer) as quantity,
  cast(revenue as numeric(12,2)) as revenue,
  trim(store_code) as store_code,
  trim(store_name) as store_name,
  trim(province) as province,
  trim(staff_code) as staff_code,
  trim(staff_name) as staff_name,
  trim(position) as position,
  nullif(trim(promo_code), '') as promo_code,
  nullif(trim(promo_desc), '') as promo_desc,
  cast(points_redeemed as integer) as points_redeemed
from {{ ref('coffee_sales') }}

```

models/staging/stg_province_region_mapping.sql

```
select
  trim(province_name) as province_name,
  trim(region_name) as region_name
from {{ ref('province_region_mapping') }}
```

ref(): {{ ref('coffee_sales') }} ไม่ได้เป็นเพียงการแทนชื่อตาราง แต่สร้าง dependency ใน lineage ทำให้ dbt รู้ว่า seed ต้องพร้อมก่อน staging model

```
dbt run --select path:models/staging
dbt show --select stg_coffee_sales --limit 5
```

ส่วนที่ 5 สร้าง Dimension Tables

โมเดลใน marts ถูกกำหนดให้ materialized เป็น table แต่ละ dimension ใช้ deterministic surrogate key จาก md5(natural_key) จึงได้ค่าเดิมเมื่อรับใหม่กับ natural key เดิม และไม่ต้องติดตั้ง dbt_utils

models/marts/dim_customer.sql

```
select distinct
  md5(customer_code) as customer_key,
  customer_code,
  customer_name,
  gender,
  birth_year
from {{ ref('stg_coffee_sales') }}
```

models/marts/dim_category.sql

```
select distinct
  md5(category) as category_key,
  category as category_name
from {{ ref('stg_coffee_sales') }}
```

models/marts/dim_product.sql

```
select distinct
  md5(s.product_code) as product_key,
  s.product_code,
  s.product_name,
  c.category_key,
  s.size,
  s.unit_price
from {{ ref('stg_coffee_sales') }} s
join {{ ref('dim_category') }} c
  on s.category = c.category_name
```

models/marts/dim_region.sql

```
select distinct
  md5(region_name) as region_key,
  region_name
from {{ ref('stg_province_region_mapping') }}
```

models/marts/dim_province.sql

```
select distinct
  md5(m.province_name) as province_key,
  m.province_name,
  r.region_key
from {{ ref('stg_province_region_mapping') }} m
join {{ ref('dim_region') }} r
  on m.region_name = r.region_name
```

models/marts/dim_store.sql

```
select distinct
  md5(s.store_code) as store_key,
  s.store_code,
  s.store_name,
  p.province_key
from {{ ref('stg_coffee_sales') }} s
join {{ ref('dim_province') }} p
  on s.province = p.province_name
```

models/marts/dim_position.sql

```
select distinct
  md5(position) as position_key,
  position as position_name
from {{ ref('stg_coffee_sales') }}
```

models/marts/dim_staff.sql

```
select distinct
  md5(s.staff_code) as staff_key,
  s.staff_code,
  s.staff_name,
  p.position_key
from {{ ref('stg_coffee_sales') }} s
join {{ ref('dim_position') }} p
on s.position = p.position_name
```

models/marts/dim_promotion.sql

```
select distinct
  md5(promo_code) as promo_key,
  promo_code,
  promo_desc
from {{ ref('stg_coffee_sales') }}
where promo_code is not null
```

models/marts/dim_date.sql

```
select distinct
  cast(to_char(sale_date, 'YYYYMMDD') as integer) as date_key,
  sale_date,
  extract(year from sale_date)::integer as year,
  extract(month from sale_date)::integer as month_number,
  trim(to_char(sale_date, 'Month')) as month_name,
  extract(quarter from sale_date)::integer as quarter,
  extract(day from sale_date)::integer as day_of_month
from {{ ref('stg_coffee_sales') }}
```

ลำดับการสร้าง: dim_product อ้าง ref('dim_category'), dim_province อ้าง ref('dim_region'), dim_store อ้าง ref('dim_province') และ dim_staff อ้าง ref('dim_position') dbt จึงสร้าง parent dimension ก่อน child dimension โดยอัตโนมัติ

5.1 รันเฉพาะ dimensions

```
dbt run --select "dim_*.sql"
```

ส่วนที่ 6 สร้าง Fact Table**models/marts/fct_sales.sql**

```
select
  s.sale_id,
  s.invoice_number,
  d.date_key,
  c.customer_key,
  p.product_key,
  st.store_key,
  sf.staff_key,
  pr.promo_key,
  s.quantity,
  s.revenue,
  s.points_redeemed
from {{ ref('stg_coffee_sales') }} s
join {{ ref('dim_date') }} d
on s.sale_date = d.sale_date
join {{ ref('dim_customer') }} c
on s.customer_code = c.customer_code
join {{ ref('dim_product') }} p
on s.product_code = p.product_code
join {{ ref('dim_store') }} st
on s.store_code = st.store_code
join {{ ref('dim_staff') }} sf
on s.staff_code = sf.staff_code
left join {{ ref('dim_promotion') }} pr
on s.promo_code = pr.promo_code
```

Promotion ที่เป็น NULL: ใช้ LEFT JOIN กับ dim_promotion เพื่อรักษาการขายที่ไม่มีโปรโมชั่นไว้ใน fact table โดย promo_key จะเป็น NULL เช่นเดียวกับแนวทางใน Lab เดิม

```
dbt run --select fct_sales
dbt show --select fct_sales --limit 10
```

ส่วนที่ 7 สร้าง hierarchy_def และ Reporting View

7.1 Metadata model

models/metadata/hierarchy_def.sql

```
with hierarchy as (
  select *
  from (values
    ('geo', 1, 'region', 'dim_region',
     'region_key', 'region_name'),
    ('geo', 2, 'province', 'dim_province',
     'province_key', 'province_name')
  ) as h(
    hierarchy_name,
    level_num,
    level_name,
    table_name,
    key_field,
    name_field
  )
)
select * from hierarchy
```

level_num กำหนดลำดับจากระดับบนลงล่าง คือ region (1) → province (2) ส่วน table_name, key_field และ name_field บอกตำแหน่งของข้อมูลใน schema

7.2 Flattened view สำหรับ Metabase

models/reporting/v_sales_geo_flat.sql

```
select
  d.sale_date,
  f.invoice_number,
  p.province_name,
  r.region_name,
  f.revenue
from {{ ref('fct_sales') }} f
join {{ ref('dim_date') }} d
  on f.date_key = d.date_key
join {{ ref('dim_store') }} s
  on f.store_key = s.store_key
join {{ ref('dim_province') }} p
  on s.province_key = p.province_key
join {{ ref('dim_region') }} r
  on p.region_key = r.region_key
```

เหตุผลที่ยังต้องมี flat view: Snowflake Schema เหมาะกับการจัดเก็บและควบคุมความซ้ำซ้อน แต่ BI tool ใช้งานง่ายขึ้นเมื่อมี reporting view ที่ join hierarchy ให้แล้ว

```
dbt run --select hierarchy_def v_sales_geo_flat
dbt show --select hierarchy_def
```

ส่วนที่ 8 กำหนด Data Tests

ไฟล์ schema.yml ระบุ uniqueness, not_null และ relationships tests สำหรับคีย์หลักและคีย์อ้างอิง ส่วน singular tests ตรวจสอบเงื่อนไขทางธุรกิจที่ต้องคืนค่า 0 แถวจึงจะผ่าน

models/schema.yml

```

version: 2

models:
- name: stg_coffee_sales
  description: Cleaned sales rows; grain is one sale_id.
  columns:
    - name: sale_id
      tests: [not_null, unique]
    - name: customer_code
      tests: [not_null]
    - name: product_code
      tests: [not_null]
    - name: store_code
      tests: [not_null]
    - name: staff_code
      tests: [not_null]
    - name: sale_date
      tests: [not_null]

- name: stg_province_region_mapping
  description: Province-to-region mapping used by the geography hierarchy.
  columns:
    - name: province_name
      tests: [not_null, unique]
    - name: region_name
      tests: [not_null]

- name: dim_customer
  description: Customer dimension.
  columns:
    - name: customer_key
      tests: [not_null, unique]
    - name: customer_code
      tests: [not_null, unique]

- name: dim_category
  description: Top level of the product hierarchy.
  columns:
    - name: category_key
      tests: [not_null, unique]
    - name: category_name
      tests: [not_null, unique]

- name: dim_product
  description: Product dimension linked to dim_category.
  columns:
    - name: product_key
      tests: [not_null, unique]
    - name: product_code
      tests: [not_null, unique]
    - name: category_key
      tests:
        - not_null
        - relationships:
            to: ref('dim_category')
            field: category_key

- name: dim_region
  description: Top level of the geography hierarchy.
  columns:
    - name: region_key
      tests: [not_null, unique]
    - name: region_name
      tests: [not_null, unique]

- name: dim_province
  description: Province level linked to dim_region.
  columns:
    - name: province_key
      tests: [not_null, unique]
    - name: province_name
      tests: [not_null, unique]
    - name: region_key
      tests:
        - not_null
        - relationships:
            to: ref('dim_region')
            field: region_key

- name: dim_store
  description: Store dimension linked to dim_province.
  columns:
    - name: store_key
      tests: [not_null, unique]
    - name: store_code
      tests: [not_null, unique]
    - name: province_key

```



```

tests:
  - not_null
  - relationships:
    to: ref('dim_province')
    field: province_key

- name: dim_position
description: Top level of the staff hierarchy.
columns:
  - name: position_key
    tests: [not_null, unique]
  - name: position_name
    tests: [not_null, unique]

- name: dim_staff
description: Staff dimension linked to dim_position.
columns:
  - name: staff_key
    tests: [not_null, unique]
  - name: staff_code
    tests: [not_null, unique]
  - name: position_key
    tests:
      - not_null
      - relationships:
        to: ref('dim_position')
        field: position_key

- name: dim_promotion
description: Promotion dimension; non-promotion facts keep a null promo_key.
columns:
  - name: promo_key
    tests: [not_null, unique]
  - name: promo_code
    tests: [not_null, unique]

- name: dim_date
description: Calendar date dimension.
columns:
  - name: date_key
    tests: [not_null, unique]
  - name: sale_date
    tests: [not_null, unique]

- name: fct_sales
description: Sales fact; grain is one sale_id.
columns:
  - name: sale_id
    tests: [not_null, unique]
  - name: date_key
    tests:
      - not_null
      - relationships:
        to: ref('dim_date')
        field: date_key
  - name: customer_key
    tests:
      - not_null
      - relationships:
        to: ref('dim_customer')
        field: customer_key
  - name: product_key
    tests:
      - not_null
      - relationships:
        to: ref('dim_product')
        field: product_key
  - name: store_key
    tests:
      - not_null
      - relationships:
        to: ref('dim_store')
        field: store_key
  - name: staff_key
    tests:
      - not_null
      - relationships:
        to: ref('dim_staff')
        field: staff_key
  - name: promo_key
    tests:
      - relationships:
        to: ref('dim_promotion')
        field: promo_key
  - name: revenue
    tests: [not_null]

- name: hierarchy_def
description: Metadata that defines the order and fields of hierarchies.
columns:

```

```
- name: hierarchy_name
  tests: [not_null]
- name: level_num
  tests: [not_null]
- name: level_name
  tests: [not_null]

- name: v_sales_geo_flat
  description: Flattened geography view for Metabase.
  columns:
    - name: sale_date
      tests: [not_null]
    - name: province_name
      tests: [not_null]
    - name: region_name
      tests: [not_null]
    - name: revenue
      tests: [not_null]
```

tests/assert_all_provinces_mapped.sql

```
select s.*
from {{ ref('stg_coffee_sales') }} s
left join {{ ref('dim_province') }} p
  on s.province = p.province_name
where p.province_key is null
```

tests/assert_revenue_nonnegative.sql

```
select *
from {{ ref('fct_sales') }}
where revenue < 0
```

dbt เรียก singular test เมื่อใด: เมื่อรัน dbt test หรือ dbt build dbt จะค้นหาไฟล์ .sql ใน test-paths ซึ่งกำหนดเป็น tests แล้วนำ query ไปตรวจ ถ้า query ค้นอย่างน้อย 1 แถว test จะ fail

```
dbt test
```

8.1 รันทั้ง pipeline ด้วยคำสั่งเดียว

```
dbt build
```

dbt build จะจัดลำดับ seed, model และ test ตาม dependency graph ที่เกิดจาก ref() โดยอัตโนมัติ

8.2 ตรวจสอบจำนวนแถว

```
select 'dim_customer' as model, count(*) from dbt_marts.dim_customer
union all select 'dim_category', count(*) from dbt_marts.dim_category
union all select 'dim_product', count(*) from dbt_marts.dim_product
union all select 'dim_region', count(*) from dbt_marts.dim_region
union all select 'dim_province', count(*) from dbt_marts.dim_province
union all select 'dim_store', count(*) from dbt_marts.dim_store
union all select 'dim_position', count(*) from dbt_marts.dim_position
union all select 'dim_staff', count(*) from dbt_marts.dim_staff
union all select 'dim_promotion', count(*) from dbt_marts.dim_promotion
union all select 'dim_date', count(*) from dbt_marts.dim_date
union all select 'fct_sales', count(*) from dbt_marts.fct_sales
order by model;
```

Model	จำนวนแถว	Model	จำนวนแถว
dim_customer	2,745	dim_category	3
dim_product	5	dim_region	3
dim_province	3	dim_store	3
dim_position	3	dim_staff	4
dim_promotion	2	dim_date	31
fct_sales	3,000		

ส่วนที่ 9 สร้าง dbt Documentation และตรวจ Lineage

```
dbt docs generate
dbt docs serve --host 0.0.0.0 --port 8080 --no-browser
```

เปิด <http://localhost:28088> แล้วเลือก fct_sales, dim_store หรือ v_sales_geo_flat จากนั้นดู Lineage Graph

- dbt docs generate อ่าน manifest, catalog และคำอธิบายใน schema.yml แล้วสร้างเว็บไซต์เอกสารแบบ static
- Lineage แสดง dependency จาก seed → staging → dimensions/fact → reporting view
- Column tests และคำอธิบายโมเดลช่วยให้ตรวจสอบ data contract ในระดับพื้นฐานได้

เมื่อแก้ไขโมเดล: ให้รัน dbt docs generate ใหม่เพื่ออัปเดต catalog และ lineage ก่อน refresh หน้าเว็บ

ส่วนที่ 10 สร้าง Dashboard บน Metabase

10.1 เชื่อมต่อ PostgreSQL

เปิด <http://localhost:23000> แล้วเพิ่ม PostgreSQL database ด้วยค่าต่อไปนี้

รายการ	ค่า
Display name	coffee_dw_snowflake
Host	postgres
Port	5432
Database name	coffee_dw_snowflake
Username	dw_user
Password	dw_pass

พอร์ต: localhost:23000 ใช้เปิดหน้า Metabase จากเครื่องผู้เรียน แต่ Metabase เชื่อม PostgreSQL ภายใน Docker network ที่ port 5432

10.2 สร้างคำถาม Revenue by Hierarchy Level

เลือก New → SQL query → database coffee_dw_snowflake แล้ววาง SQL ต่อไปนี้

```
with selected_level as (
  select level_name
  from dbt_metadata.hierarchy_def
  where hierarchy_name = 'geo'
  [[and {{level_name}}]]
  order by level_num
  limit 1
)
select
  case sl.level_name
    when 'province' then f.province_name
    when 'region' then f.region_name
  end as selected_level,
  sum(f.revenue) as total_revenue
from dbt_reporting.v_sales_geo_flat f
cross join selected_level sl
group by 1
order by 2 desc
```

ตั้งค่า variable level_name เป็น Field Filter แล้ว map ไปที่ dbt_metadata.hierarchy_def.level_name จากนั้นเลือก widget เป็น Dropdown list

เมื่อยังไม่เลือก filter query จะใช้ region ซึ่งเป็น level_num ต่ำสุด เมื่อเลือก province จะแสดงรายได้แยกจังหวัด

สิ่งที่ต้องส่ง

รายการ	รูปแบบ
Screenshot หน้า dbt Lineage ของ v_sales_geo_flat	PNG
Screenshot ผล dbt test ที่ผ่านทั้งหมด	PNG
Screenshot ผลของ Revenue by Hierarchy Level ที่สลับ region / province ได้	PNG